Taller 3 - Diseño

CarPooling UPB

Documento de diseño técnico y prototipado

Samuel Delgado Zapata

Samuel Aguilar Ruiz

Juan José Lastra Hernández

Sofía Duque Gallo

Universidad Pontificia Bolivariana
Ingeniería de Sistemas e Informática
Medellín
Septiembre de 2026

1. Documento de diseño técnico

Este documento presenta las decisiones de diseño de CarPooling UPB a partir de los requisitos consolidados en el Taller 2. El objetivo es definir una arquitectura sencilla, coherente con el alcance académico del proyecto y fácil de implementar por etapas, sin incorporar funcionalidades que fueron clasificadas fuera del alcance.

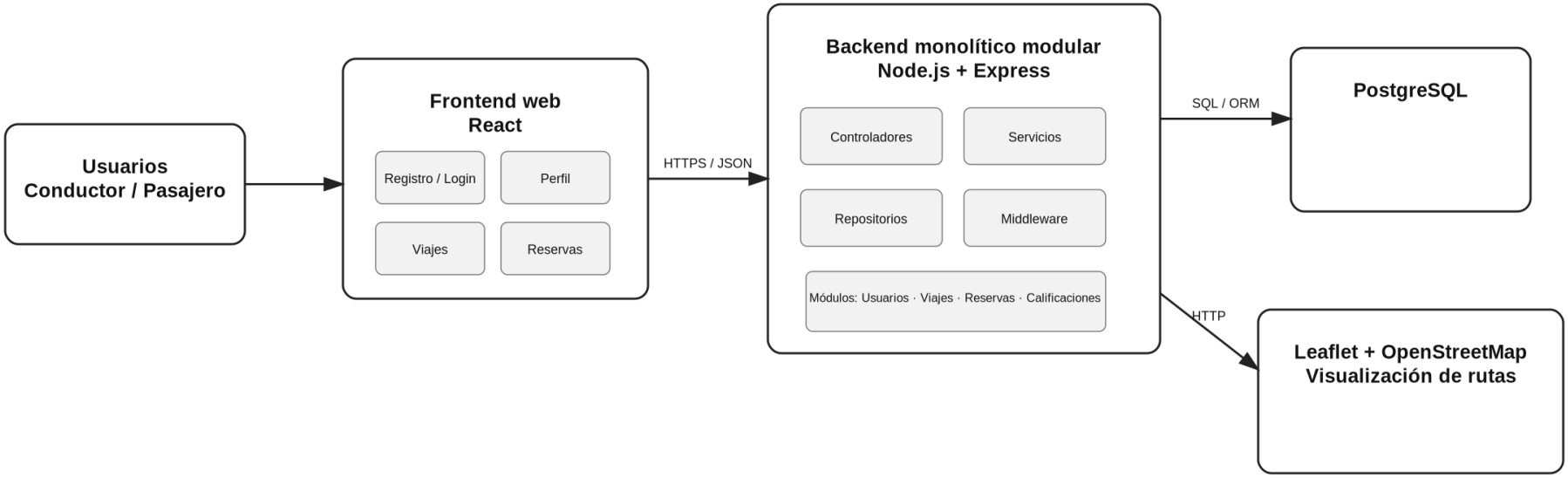
1.1 Arquitectura de software

Se propone una arquitectura cliente-servidor con un backend monolítico modular organizado por capas. La interfaz web se comunica con una API REST y el backend centraliza autenticación, reglas de negocio, acceso a datos e integración con el servicio de mapas.

La elección de un monolito modular responde al tamaño actual del proyecto: los módulos de usuarios, viajes, reservas y calificaciones están estrechamente relacionados y pueden mantenerse dentro de una sola aplicación backend sin introducir la complejidad operativa de microservicios. La modularidad permite separar responsabilidades y facilitar una futura evolución si el sistema crece.

- Cliente web: presenta la interfaz y gestiona la interacción del estudiante conductor o pasajero.
- API REST: recibe solicitudes del frontend y entrega respuestas en formato JSON.
- Backend: aplica reglas de negocio, autenticación, autorización y validaciones.
- Base de datos: almacena usuarios, viajes, reservas y calificaciones.
- Servicio de mapas: permite visualizar rutas de forma básica, sin rastreo GPS en tiempo real.

Arquitectura propuesta



Estructura interna del backend

El backend se organiza por módulos funcionales y por capas. Cada módulo agrupa las reglas y operaciones relacionadas con una parte del sistema, mientras que las capas mantienen separadas las responsabilidades técnicas.

Capa	Responsabilidad
Rutas / Controladores	Reciben las solicitudes HTTP, validan datos básicos y retornan la respuesta al frontend.
Servicios	Contienen la lógica del negocio: publicación de viajes, reservas, cupos, calificaciones y reglas de acceso.
Repositorios	Concentran las operaciones de lectura y escritura sobre la base de datos.
Modelos	Representan entidades como Usuario, Viaje, Reserva y Calificación.
Middleware	Gestiona autenticación, autorización, manejo de errores y validaciones comunes.

Módulos funcionales: Autenticación, Usuarios, Viajes, Reservas y Calificaciones.

Estructura interna del frontend

El frontend se divide en páginas, componentes reutilizables y servicios de comunicación con la API. Las principales vistas corresponden directamente al flujo del usuario: registro e inicio de sesión, perfil, publicación de viaje, búsqueda, resultados, detalle, reserva y gestión de viajes/reservas.

- Páginas: Login, Registro, Inicio, Perfil, Publicar viaje, Buscar, Resultados, Detalle, Reserva confirmada y Mis viajes.
- Componentes: barra de navegación, tarjetas de viaje, formularios, filtros, indicadores de cupos y calificación.
- Servicios: funciones para consumir la API y mantener separada la lógica de comunicación de la interfaz visual.

1.2 Stack tecnológico

El stack seleccionado mantiene una tecnología homogénea en frontend y backend, favorece el desarrollo web por componentes y se ajusta al alcance del proyecto.

Componente	Tecnología	Justificación
Frontend	React + TypeScript	Permite construir vistas por componentes reutilizables y mantener tipado el código de la interfaz.
Backend	Node.js + Express + TypeScript	Facilita la construcción de una API REST y mantiene el mismo lenguaje base en frontend y backend.
Base de datos	PostgreSQL	El dominio tiene relaciones

		claras entre usuarios, viajes, reservas y calificaciones, por lo que un modelo relacional es adecuado.
Acceso a datos	Prisma ORM	Simplifica el modelado de entidades y las operaciones con PostgreSQL desde el backend.
Mapas	Leaflet + OpenStreetMap	Permite representar rutas y puntos sin incorporar rastreo GPS en tiempo real ni depender de una solución propietaria de pago.
Control de versiones	Git + GitHub	Permite trabajo colaborativo, control de cambios y respaldo del código.
Prototipado	Figma Make / Figma	Permite representar el flujo visual y validar la navegación antes de implementar la interfaz.

Comunicación entre componentes. El frontend consume el backend mediante HTTPS y JSON. El backend utiliza Prisma para acceder a PostgreSQL y realiza las consultas al servicio de mapas únicamente cuando se requiere representar la ruta.

2. Prototipado

El prototipado se desarrolló en dos niveles. Primero se definió el flujo mediante wireframes de baja fidelidad y luego se creó un prototipo de media fidelidad con identidad visual, componentes y navegación coherentes con los requisitos del sistema.

2.1 Prototipado de baja fidelidad

Los wireframes muestran la organización general de las pantallas sin enfocarse en detalles gráficos. El flujo principal del pasajero parte del inicio de sesión, continúa con la búsqueda y consulta de resultados, pasa al detalle y termina con la reserva. El flujo del conductor permite publicar un viaje y gestionarlo posteriormente.

Wireframes de baja fidelidad



2.2 Prototipado de media fidelidad

A partir de los wireframes se construyó un prototipo de media fidelidad en Figma Make. Se definió una identidad visual basada en tonos azules para la navegación principal, verde para confirmaciones y rojo para acciones de cancelación. Se priorizó la comprensión del flujo, la lectura de la información y la consistencia entre pantallas.

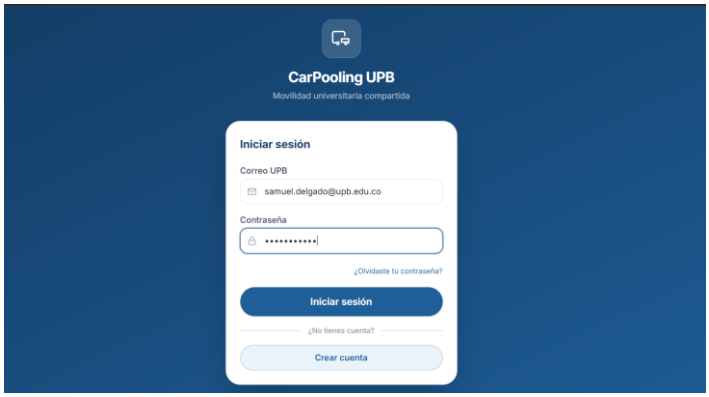
Criterios aplicados:

- Uso de correo institucional UPB como referencia de pertenencia a la comunidad.
- Búsqueda por origen, destino, fecha y horario.
- Visualización de cupos, punto de encuentro y calificación del conductor.
- Reserva de cupo y gestión de viajes/reservas.
- Uso de mapa o ruta únicamente como representación visual, sin seguimiento GPS en tiempo real.
- No se incorporan pagos integrados, chat interno ni verificación exhaustiva de documentos.

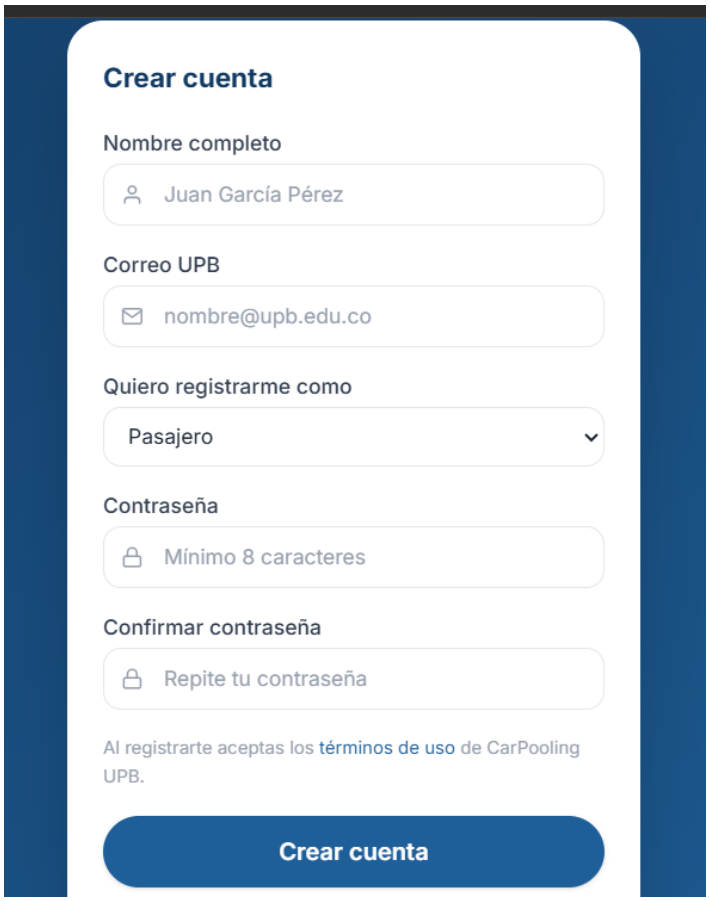
Prompt utilizado para generar la base del prototipo:

> Diseña un prototipo web responsive de media fidelidad para CarPooling UPB, una plataforma para estudiantes de la UPB Medellín donde conductores publican viajes y pasajeros buscan y reservan cupos. Estilo moderno, limpio, universitario y confiable. Usar azul oscuro #123B63, azul #1F5F99, azul claro #EAF3FA, blanco y gris claro; verde para confirmaciones y rojo para cancelaciones. Crear pantallas de Login, Registro, Inicio, Perfil, Publicar viaje, Buscar viaje, Resultados, Detalle, Reserva confirmada y Mis viajes/reservas. Flujo pasajero: Login → Inicio → Buscar → Resultados → Detalle → Reservar → Confirmación. Flujo conductor: Inicio → Publicar viaje → Mis viajes → Editar/Cancelar. No incluir pagos, chat interno, GPS en tiempo real ni verificación de documentos. No generar imágenes; usar placeholders e íconos.

Inicio de sesión



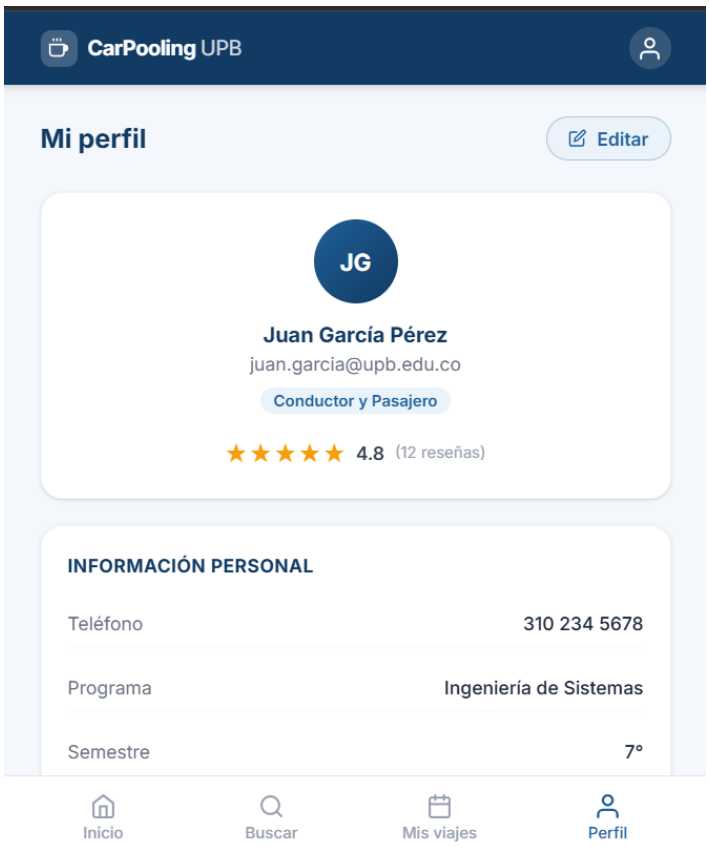
Registro



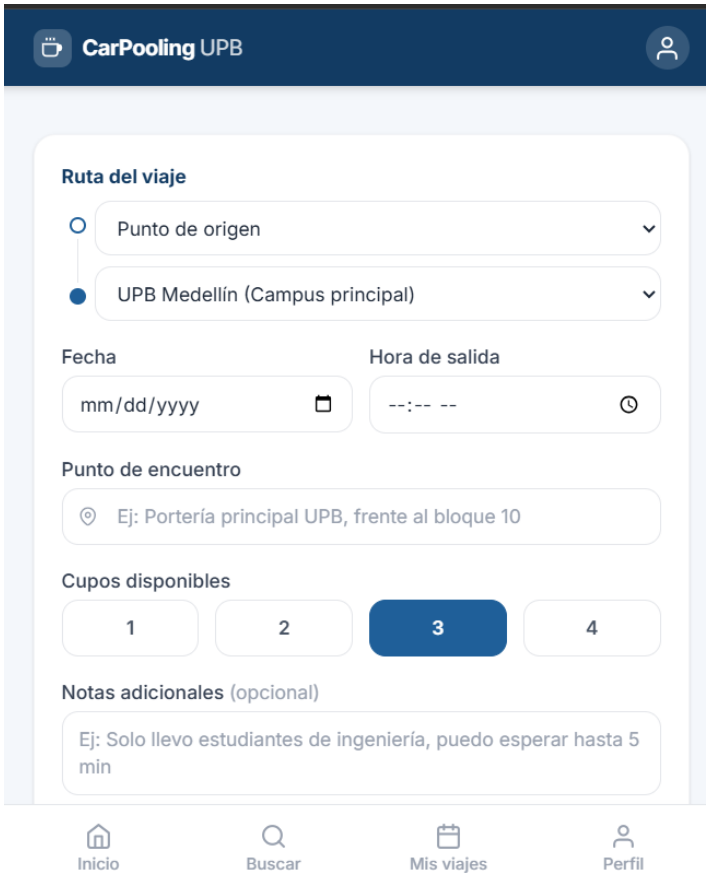
Inicio / Dashboard



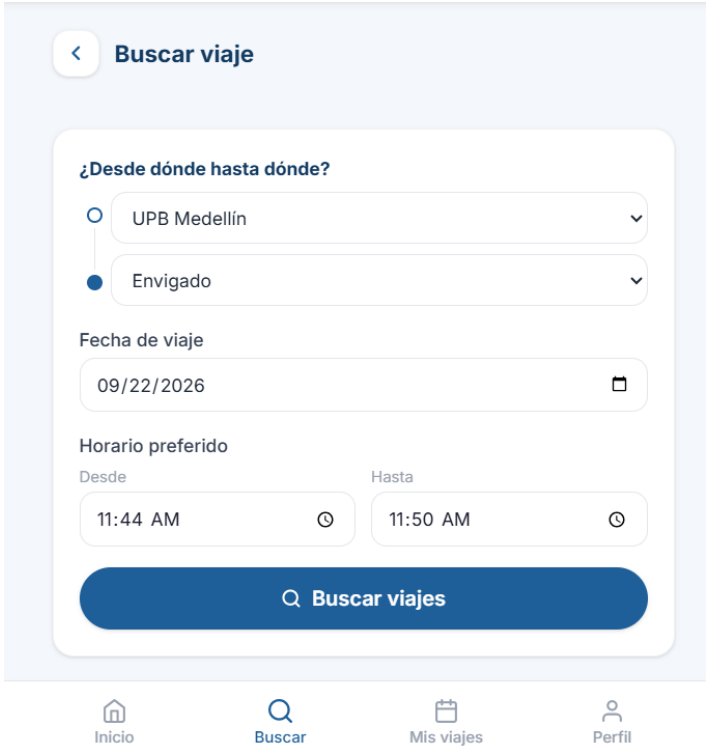
Perfil



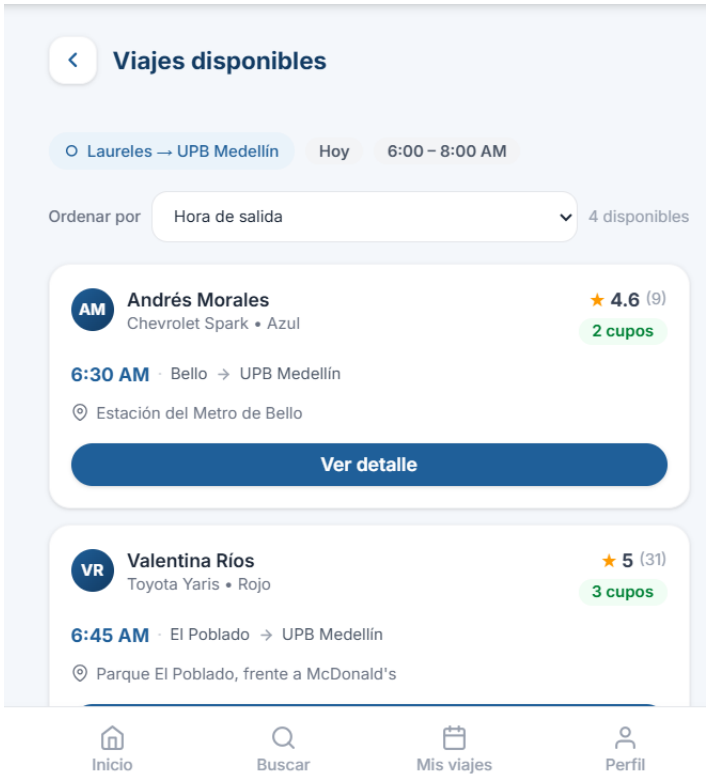
Publicar viaje



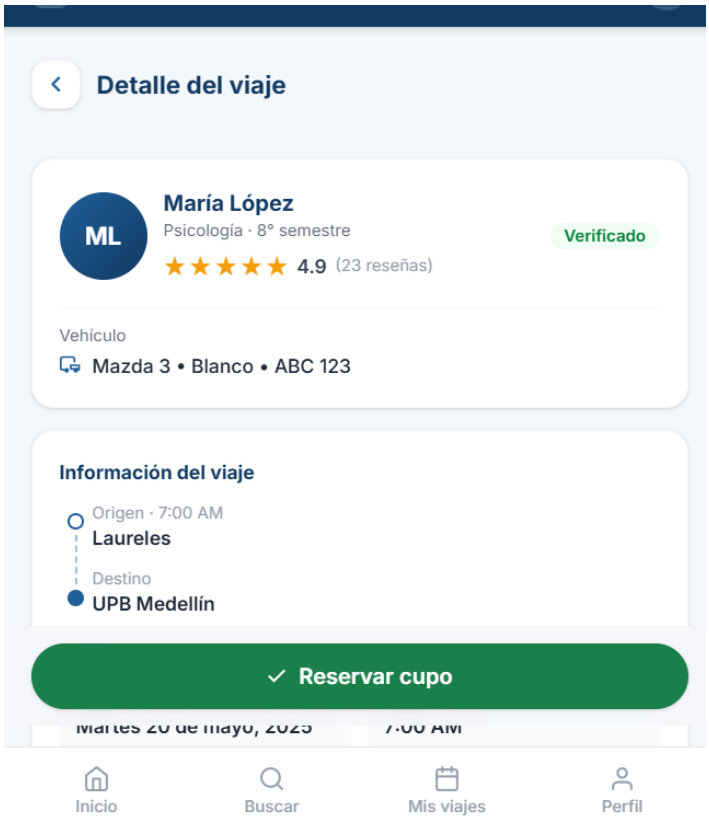
Buscar viaje



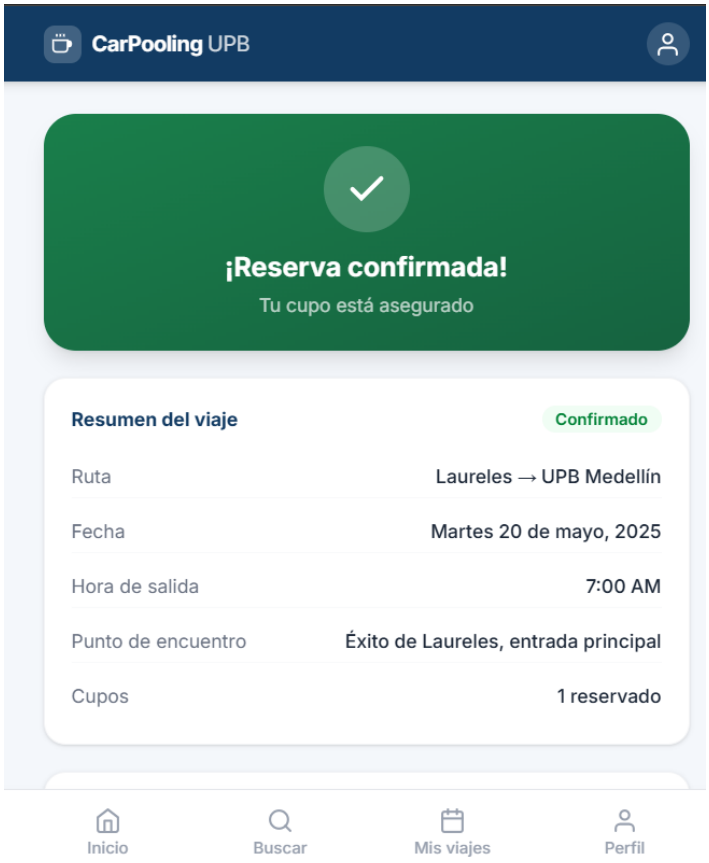
Resultados de búsqueda



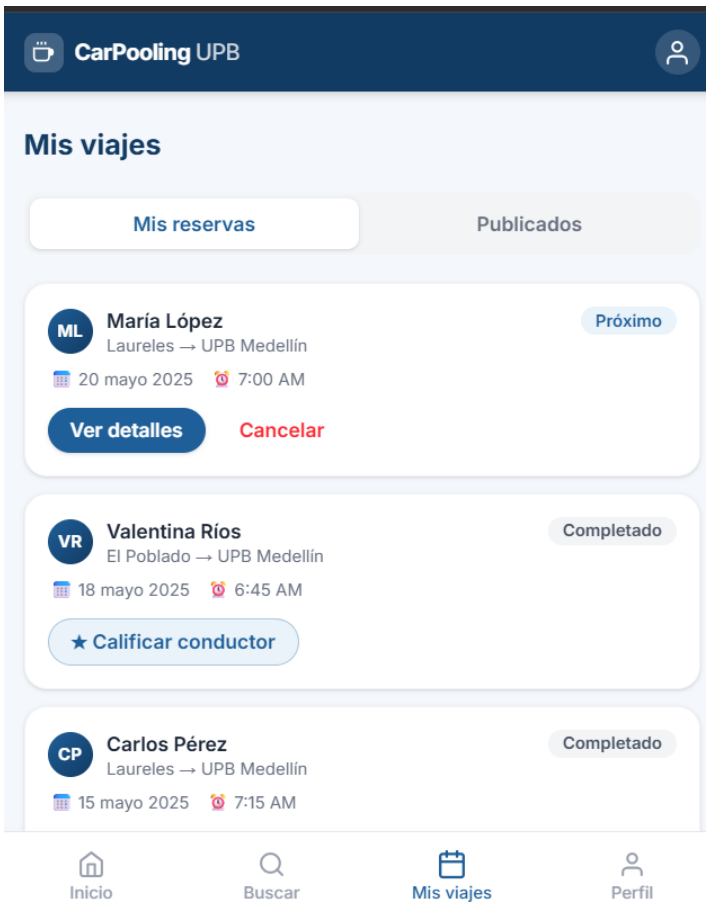
Detalle del viaje



Reserva confirmada



Mis viajes / Mis reservas



3. Resultado del diseño

La propuesta mantiene correspondencia entre requisitos, arquitectura y prototipado. Los componentes técnicos soportan las funciones principales definidas para esta iteración y el flujo visual permite validar la experiencia antes de iniciar la implementación. El diseño conserva fuera del alcance las funciones que requieren mayor complejidad técnica o regulatoria, como pagos integrados, chat interno, rastreo GPS en tiempo real y verificación exhaustiva de documentos.